HÁSKÓLINN Í REYKJAVÍK
REYKJAVIK UNIVERSITY

Final Project

Created by Bjarki Smári Smárason & Hildur Davíðsdóttir

Introduction

The goal of this project is to work with a legacy database that is in grave need of redesign, improve its schema and build a backend system around it using FastAPI. The assignment tests database understanding, SQL querying, schema design, backend implementation, and query performance. **Please read the instructions below carefully**. If you have any questions, do not hesitate to ask on Piazza.

This is a final project in the Databases course and is created in collaboration with The Icelandic Environment and Energy Agency.

Umhverfis-
og orkustofnun

Table of Contents

Deliverables 3
Project files 3
Presentation 3
Competition 4
Domain 5
Required Tasks 7
Part A: Analysis of the legacy database 7
Task A1: Setting up the database 7
Task A2: Write DQL queries 8
Part B: FastAPI Integration 11
Task B1: Setting up the API 11
Task B2: Expose the queries to the API 12
Part C: Refactoring the database 13
Task C1: Improve the database design 13
Task C2: Draw an ER diagram 13
Task C3: Reconstruct the database 13
Task C4: Migrate the data 13
Task C5: Replicate the three queries 13
Part D: Indexing and Performance 14
Task D1: Create an indexing strategy 14
Advanced Tasks 15
Task E: A POST endpoint 15
Task F: Substation Flow Estimation 16

Deliverables

Project files

You are provided with a folder structure which you will work with and extend throughout this project. Once you are finished, you must compress the folder (.zip) and submit it to Canvas by **midnight March 31st (Tue) at 23:59**.

The project consists of required tasks (parts A-D) as well as advanced tasks (parts E-F). It is mandatory to make a meaningful attempt at all the required tasks in order to receive a grade for this project. The advanced tasks are optional and provide further learning opportunities as well as a grade-boosting opportunity to demonstrate additional initiative and deeper understanding.

Note that task labels are marked inside the files that need editing in each task.

Presentation

The final component of this project is a final presentation to be delivered in the period of April 8-10. Presentation booking will open on Monday, March 30th at 12:00 and is done through the *Canvas Calendar => Find Appointments => Gagnasafnsfræði*. Only one member of the group books the slot and writes the group name as the booking comment.

The presentation should be 8-12 minutes long, followed by ~3-5 minutes of questions. You must have PowerPoint slides to support your presentation and must submit them to Canvas by **noon April 8th (Wed) at 12:00**. The filename of the slides must be `your_group_name.pptx`.

The presentation should demonstrate your solutions and include justification for your design decisions. It should briefly (~1 min) talk about the project management; how the group work panned out and which hurdles you had to overcome. Each member of the group should be able to answer for all parts of the project.

Note: The grade will be based on both the project file deliverables as well as the presentation delivery.

Competition

This project has a competition component, hosted by the project's creator: *The Icelandic Environment and Energy Agency*.

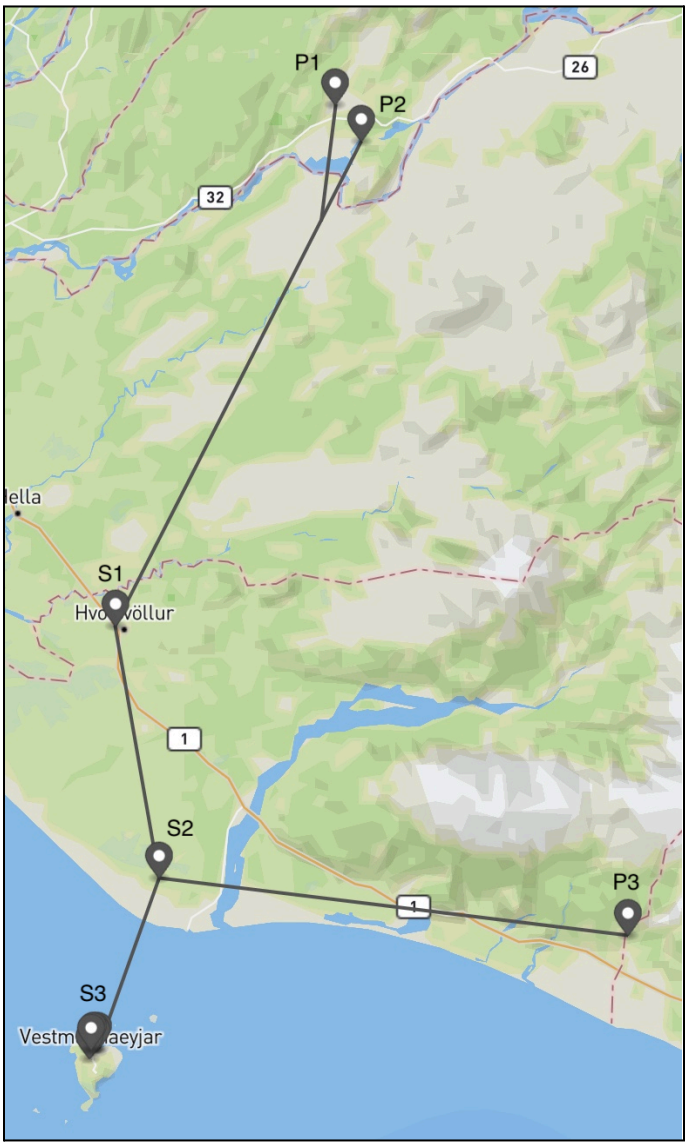
Participation in the competition is entirely optional, meaning that you can still get full marks for the project regardless of whether you choose to participate or not. The deliverables are exactly the same if you choose to take part in the competition, but your presentation will be 5 minutes longer as it will also be attended by a representative from the agency who may ask some questions as well. The agency then chooses the winning team. The prize is awarded to the group as a whole, so every member of the group must be a team player!

So how do I win this competition? Of course, the extent and quality of your implementation is the strongest factor, but there are various ways to stand out to the judges, e.g. come up with your own bonus tasks, deliver a knock-out presentation with strong reasoning, etc... Use your imagination 🤪

Note: Participation in the competition does not affect how the project is graded in the course. The project grade is determined by the course responsible, based on the same academic criteria for all groups. The agency only selects the winner of the competition.

Domain

You are given an imaginary electricity grid segment, loosely based on real-world data but fully simulated. The scenario models electricity production and transmission toward Vestmannaeyjar. The dataset represents a simplified transmission chain where energy is produced at multiple power plants, injected into substations, and finally withdrawn at the terminal node representing consumption in Vestmannaeyjar.



The simulated grid works as follows:

- Power plants P1 (Þröstur) and P2 (Búrfell) inject energy into substation S1 (Krókur).
- Power plant P3 (Strokkur) injects energy into substation S2 (Rimakot).
- All energy is ultimately withdrawn at substation S3 (Vestmannaeyjar) by end customers.
- The connection lines in the power grid represent transmission lines. P → S denotes a line from a power plant to a substation, and S → S denotes a line between substations. Connections shown are P1 → S1, P2 → S1, S1 → S2, P3 → S2, and S2 → S3.

Three types of measurements are recorded by the system:

- *Framleiðsla*: The raw energy production at a power plant.
- *Innmötun*: The energy injected into a substation after *plant-level losses*.
- *Úttekt*: The energy withdrawn at S3 by an end customer.

Electricity in a physical grid must always follow a strict energy ordering: production at the power plant is always greater than what gets injected into the substation, which in turn is always greater than what is finally withdrawn by customers. This is because losses occur at every stage, first between the plant and the substation due to transformer and connection inefficiencies, and then again along the transmission cables between substations before the energy reaches its final destination. Energy cannot be created along the way, only lost. This means Framleiðsla > Innmötun > Úttekt is not just an expectation, it is a physical law that the data must satisfy.

Note: The database does NOT record the intermediate transmission flows between substations. (Solving those missing links is the subject of one of the advanced tasks).

Required Tasks

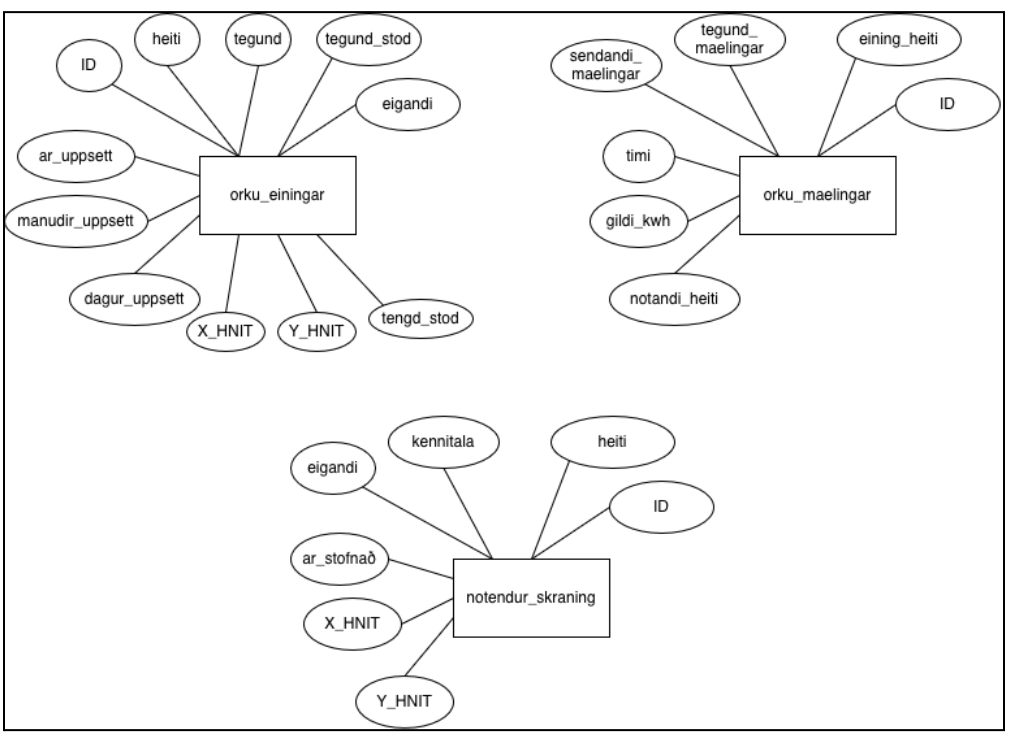
Part A: Analysis of the legacy database

Task A1: Setting up the database

You are provided with a PostgreSQL legacy database of the system, containing data for the year 2025. You must begin by creating a database called OrkuflaediIsland and populate it with the `DDL_DML.sql` file provided using `psql`. The schema of the database is intentionally poorly designed and difficult to work with.

The legacy schema:

- `orku_einingar`: A table with both power plants and substations, referenced by name strings.
- `orku_maelingar`: A table with all the power plant measurements (Framleiðsla, Innmötun, Úttekt).
- `notendur_skraning`: A table with all the end users (energy customers).
- `test_measurement`: A table used for insert performance testing.



Task A2: Write DQL queries

Write and execute the following three queries directly against the legacy schema. These must work against `raforka_legacy.orku_maelingar` as-is. You should definitely experiment with the database and familiarize yourself with the data by using `SELECT *` and the `LIMIT` statement to only preview a few rows.

Query 1: *Monthly energy flow per power plant*

For each power plant and each month in 2025, calculate the total energy measured for each of the three measurement types (Framleiðsla, Innmötun, Úttekt). The query should display the name of the power plant, the year and month being analysed, the measurement type and the total kwh measured. The records should be ordered by power plant, month (ascending) and the total_kwh (descending).

This is a preview of what you should get:

108 rows returned

	power_plant_source text	year integer	month integer	measurement_type text	total_kwh numeric
1	P1_Þröstur	2025	1	Framleiðsla	20846.048525598040398
2	P1_Þröstur	2025	1	Innmötun	20424.445664562691925
3	P1_Þröstur	2025	1	Úttekt	19813.3003170770639469
4	P1_Þröstur	2025	2	Framleiðsla	18890.060678097627688
5	P1_Þröstur	2025	2	Innmötun	18513.594170412214691
6	P1_Þröstur	2025	2	Úttekt	17958.0435766203702843
7	P1_Þröstur	2025	3	Framleiðsla	20861.306200546840149
8	P1_Þröstur	2025	3	Innmötun	20441.056515206000342

Reflection: Does the physical energy law hold? (*Framleiðsla* > *Innmötun* > *Úttekt*)

Query 2: *Monthly energy usage per customer*

For each power plant and each month in 2025, calculate the total energy withdrawn for each customer. The query should display the name of the power plant, the year and month being analysed, the customer name and the total kwh withdrawn. The records should be ordered by power plant, month (ascending) and the customer name (ascending).

This is a preview of what you should get:

180 rows returned

	power_plant_source text	year integer	month integer	customer_name text	total_kwh numeric
1	P1_Þröstur	2025	1	Herjólfur	2971.9950475615594534
2	P1_Þröstur	2025	1	Isavia	3962.6600634154130717
3	P1_Þröstur	2025	1	Laxey ehf	5943.9900951231188967
4	P1_Þröstur	2025	1	Vestmannaeyjar ehf	3962.6600634154130717
5	P1_Þröstur	2025	1	ÍBV Íþróttafélag	2971.9950475615594534
6	P1_Þröstur	2025	2	Herjólfur	2693.7065364930554210
7	P1_Þröstur	2025	2	Isavia	3591.6087153240742996
8	P1_Þröstur	2025	2	Laxey ehf	5387.4130729861108431

Reflection: Which customer consumed the most energy in 2025?

Query 3: *Monthly energy loss ratio per power plant*

For each power plant, calculate the following two loss ratios, averaged across 2025. The first ratio, `plant_to_substation_loss_ratio`, should measure the fraction of Framleiðsla lost between Framleiðsla and Innmötun. The second ratio, `total_system_loss_ratio`, should measure the fraction of Framleiðsla lost between Framleiðsla and Úttekt.

The query should use a view for the aggregation per power plant and month, and then a main query to calculate the average across the whole year.

This is a preview of what you should get:

3 rows returned

	power_plant_source text	plant_to_sub_loss_ratio numeric	total_system_loss_ratio numeric
1	P1_Þröstur	0.01995631531935981498	0.04936991085310608013
2	P2_Búrfell	0.01999604639592300767	0.04942624117863427033
3	P3_Strokkur	0.02007435920126173722	0.03474102894395639367

Part B: FastAPI Integration

Task B1: Setting up the API

In the project folder, there is a folder called "API" that contains a backend connected to the legacy database. In previous assignments, we connected Python directly to PostgreSQL using the `psycopg` library. In this project, that low-level connection layer still exists underneath, but the backend itself uses `SQLAlchemy`, which is a higher-level library for working with databases in Python and is commonly used together with APIs. Since this may be new to you, part of the task will be to explore the structure of `SQLAlchemy` and understand how it is used in the provided codebase.

Start by navigating to the Legacy folder and analysing each subfolder:

- `db`: Contains the python mapping of the PostgreSQL tables
- `models`: Contains the response models
- `parsers`: Contains the data parsers
- `routes`: Contains the HTTP endpoints
- `services`: Contains the query logic

When a call is made to an endpoint, the system flow works as follows:

main => routes => services => db => SQL => models

It is recommended that you create a virtual environment for the project so you don't have libraries all over the place. After creating and activating the environment, install the dependencies in the `requirements.txt` file using `pip`.

```
pip install -r requirements.txt
```

Before starting the API, update the `session.py` file so that the database connection settings match your local PostgreSQL setup. Once this has been configured, start the application from the correct project directory using:

```
uvicorn app.main:app --reload
```

Now a Swagger should be available at `127.0.0.1:8000/docs`. Experiment with the API by testing the endpoints that are already in place. You can use the debugger to analyse the system flow at the same time.

Note: All endpoints accept optional from_date and to_date query parameters. If not provided, they default to 2025-01-01 00:00:00 as the start date and 2026-01-01 00:00:00 as the end date, covering the full year of 2025. A date validation helper is already provided in the project to handle this. Be aware that when querying over large date ranges against the legacy schema, response times may be slow as the legacy tables have no indexes. It is recommended to test with a narrow date range first, for example a single hour or a single day, before querying the full year.

Task B2: Expose the queries to the API

Each of the three queries you made in part A must be exposed as **GET** endpoints under the router prefix /OrkuFlaediIsland.

Your task is to implement the service functions that each endpoint calls: `get_monthly_energy_flow_data()`, `get_monthly_customer_usage_data()`, and `get_monthly_plant_loss_ratios_data()`.

A date validation helper is already provided. Response models for these endpoints are already included under app/models/shared/ as Pydantic BaseModel classes.

Query 1 expected response shape:

GET `/OrkuFlaediIsland/monthly-energy-flow?from_date=2025-01-01T00:00:00&to_date=2026-01-01T00:00:00`


```
[
    {
        "power_plant_source": "P1_Þröstur",
        "measurement_type": "Framleiðsla",
        "year": 2025,
        "month": 1,
        "total_kwh": 29.677621886636725
    },
    {
        "power_plant_source": "P1_Þröstur",
        "measurement_type": "Innmötun",
        "year": 2025,
        "month": 1,
        "total_kwh": 29.090438696042114
    },
    {
        "power_plant_source": "P1_Þröstur",
        "measurement_type": "Úttekt",
        "year": 2025,
        "month": 1,
        "total_kwh": 28.275611418944511
    },
    …
]
```

Part C: Refactoring the database

As you have probably already gathered, the database structure is severely flawed. The objective of this part is to identify these issues and fix them.

Task C1: Improve the database design

- Identify any functional dependencies in the system and normalize the database.
- Create references between relations.
- Identify any inappropriate data types and fix them.
- Identify any missing information in the system and try to add that data.
    - *Hint: What information could be a factor in the plant loss?*

Task C2: Draw an ER diagram

Draw an ER-diagram demonstrating your new and improved design. The ER-diagram must follow the notation presented in this course.

Task C3: Reconstruct the database

Create the new database schema with your proposed fixes.

Task C4: Migrate the data

Once your new database schema is in place, you must migrate the data from the legacy database into the new one in a safe manner. All 2025 measurements from `raforka_legacy` should be present and queryable in your new schema.

Task C5: Replicate the three queries

Replicate the three queries you wrote in Task A2 and reconstruct the endpoints you created in Task B2.

Part D: Indexing and Performance

Once you have successfully refactored the database, it is time to think about performance. There is a large volume of time-series data in the database revolving around the measurements.

Task D1: Create an indexing strategy

Your schema must include an indexing strategy that supports the three queries.

Advanced Tasks

Note: These tasks are not chronological. If you have time for one advanced task, you can choose either one of them as they do not depend on each other.

Part E: A POST endpoint

Task E1: Create a POST endpoint

There must be a simple way to insert new measurements data. You must implement a **POST** endpoint at **POST** `/<your_db_name>/measurements-data` that accepts a CSV file and inserts the data correctly into your normalized database.

A sample CSV file is provided with the project containing data for the year 2026, the year following the existing dataset. The structure of each row in that CSV matches the flat row structure of orku_maelingar in the legacy database, meaning each row contains a power plant name, a measurement type, a sender reference, a timestamp, an energy value, and optionally a customer name.

Your job is to parse that CSV and map each row correctly into your new normalized schema by resolving the text references into the appropriate foreign key IDs before inserting. The project already includes a working example of this pattern in the test insert endpoint.

It is strongly recommended that you follow the same structure for your implementation. That means placing your CSV parser function inside `app/parsers/` and defining a parsed data model for a single row inside `app/models/parsed_data/`, following the same convention as the legacy example at `app/models/parsed_data/test_measurement_data.py`. Your service function should then read the file, decode it, pass the raw text to your parser, and use the resulting list of parsed row objects to build the insert payload for your measurements table.

Part F: Substation Flow Estimation

This task is closely tied to how well your new schema is designed. A clean normalized schema makes this significantly easier to solve.

The measurements table records what was produced, what was injected into substations, and what was withdrawn at S3. What it does NOT record is how much power flowed between substations. The intermediate links S1 to S2 and S2 to S3 are missing from the data entirely. Your task is to estimate these missing transmission flows and their associated losses using only the data that is available. The endpoint should accept the same optional from_date and to_date parameters with the same defaults as all other endpoints in the project.

From the measurements table you have the total injection at S1 from P1 and P2, the total injection at S2 from P3, and the total withdrawal at S3 across all customers. The substation data includes the coordinates of each substation, but cable distances between them are not provided directly. You will need to calculate the distance between each connected pair of substations from their coordinates before you can solve the flow problem. This is worth keeping in mind when designing your schema in Part 2. A table that explicitly models the connections between substations and stores the calculated distance between them will make this task significantly easier to approach.

From the distances and measurements you can derive that total system loss equals total injection minus total withdrawal. That loss happened somewhere on the cables, the question is how to split it between segments and work out what flowed where.
Hint: The missing values can be derived from what is already in the database using conservation of energy. Note that the maximum capacity of S2 to S3 cannot be accurately determined until the flow and losses on S1 to S2 have been solved first, since S2 receives power from both S1 and P3. It is therefore acceptable to leave max_capacity_mw as null or zero initially, solve all other fields in the substation flow endpoint first, and then populate the capacity values afterwards using SQL update statements.